The Last Of Us
Apollo Payne
Chris Peters
Calista Martin
Fiza Rahman

Project Overview

The project's purpose is to act as a patient record database system, with the goal of keeping patient data in a system that can be easily accessed and navigated by both the patient and any relevant medical staff. Ideally this project would also lay the framework down for further development into a fully-fledged application, incorporating legal staff into the process to navigate legally binding restrictions to medical care such as DNR, DNI, and AND requests from the patient, as well as allowing multiple medical practitioners from differing fields to make contributions to the patient's chart.

Functional Requirements

The Patient Chart Portal system implements several functional requirements to support healthcare management operations:

- Store patient demographic and medical information
- Store doctor contact and specialty information
- Allow each patient to have a secure portal account
- Schedule and manage appointments
- Support many-to-many relationships between appointments and doctors
- Store prescriptions, diagnoses, and treatment information
- Maintain patient medical records and medical chart summaries
- Retrieve healthcare information through advanced SQL queries
- Automatically log appointment activity using triggers
- Prevent deletion of patients with scheduled appointments
- Update medical records automatically when entries are modified
- Generate patient health summaries through a stored procedure

The database also demonstrates data retrieval using joins, aggregation functions, filtering, DISTINCT statements, ORDER BY clauses, string concatenation, and subqueries.

Non-Functional Requirements

Several non-functional requirements were implemented to ensure database quality, integrity, and reliability.

Data Integrity

The database uses:

- Primary Keys
- Foreign Keys
- UNIQUE constraints
- NOT NULL constraints
- DEFAULT values

These constraints prevent duplicate records, enforce valid relationships, and ensure required data is always entered correctly.

Security and Reliability

Portal accounts use hashed password fields for login credential storage. The system also uses triggers to automatically track important changes and maintain consistency between related tables.

Scalability

The database structure was normalized to reduce redundancy and improve scalability. Bridge tables such as AppointmentDoctor support flexible many-to-many relationships between entities.

Maintainability

The schema uses organized table structures, meaningful naming conventions, and modular SQL sections to improve readability and future maintenance.

Business Rules

The following business rules govern the database system:

1. Each patient must have exactly one portal account.
2. Each patient is assigned one primary doctor.
3. Doctors may participate in multiple appointments.
4. Appointments may involve multiple doctors.
5. Duplicate appointments for the same patient at the same date and time are not allowed.
6. Patient email addresses and doctor email addresses must be unique.
7. Patients with scheduled appointments cannot be deleted.
8. Medical record timestamps must automatically update when related entries are modified.
9. Every new appointment must be logged in the audit system.
10. Prescriptions and diagnoses must be linked to valid medical record entries.

These business rules are enforced through relationships, constraints, and automation features such as triggers.

Data Sources

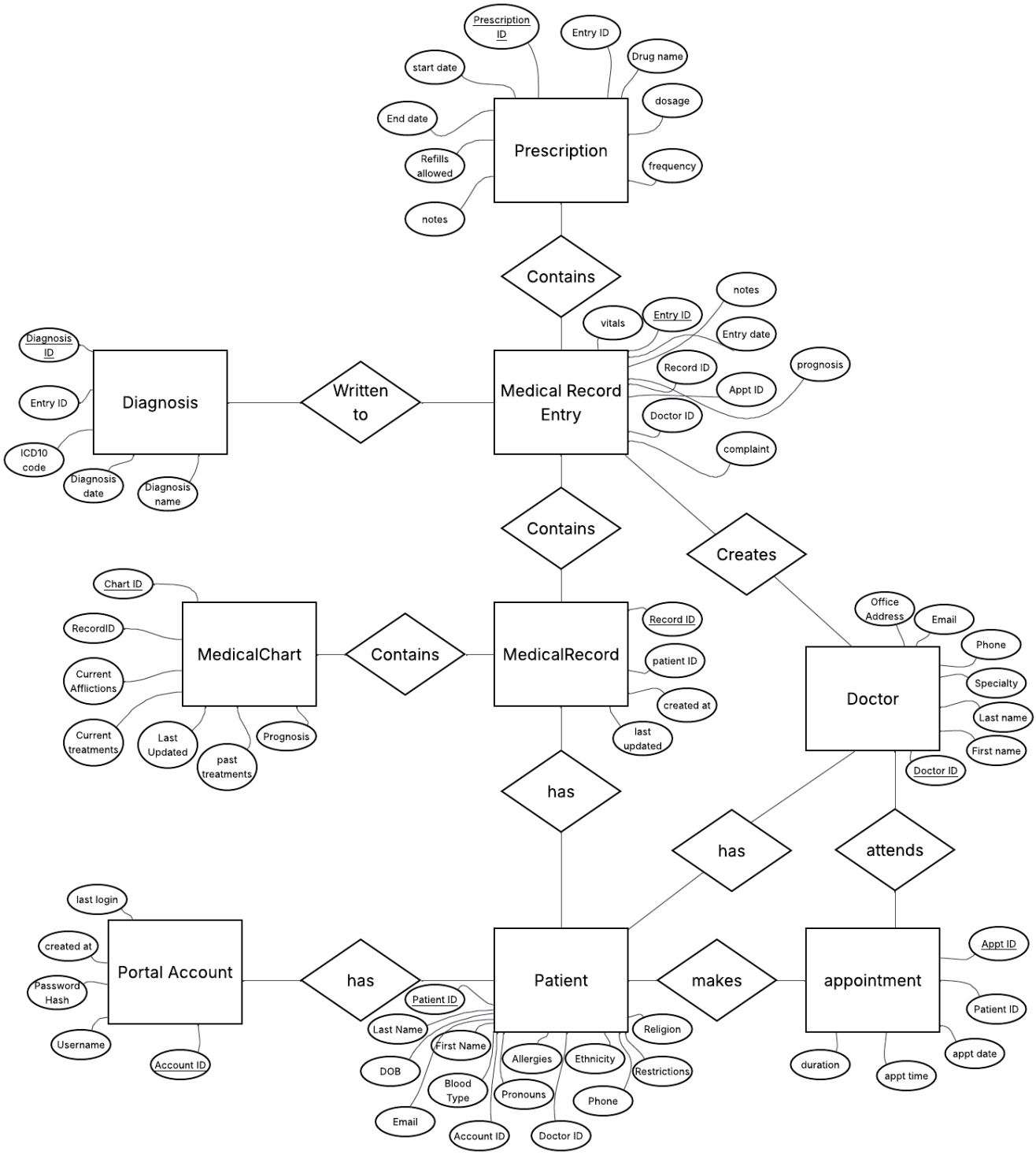
The project uses realistic manually generated sample healthcare data. The dataset includes:

- Fictional patients
- Fictional doctors
- Medical diagnoses
- Prescription records
- Appointment histories
- Medical chart summaries

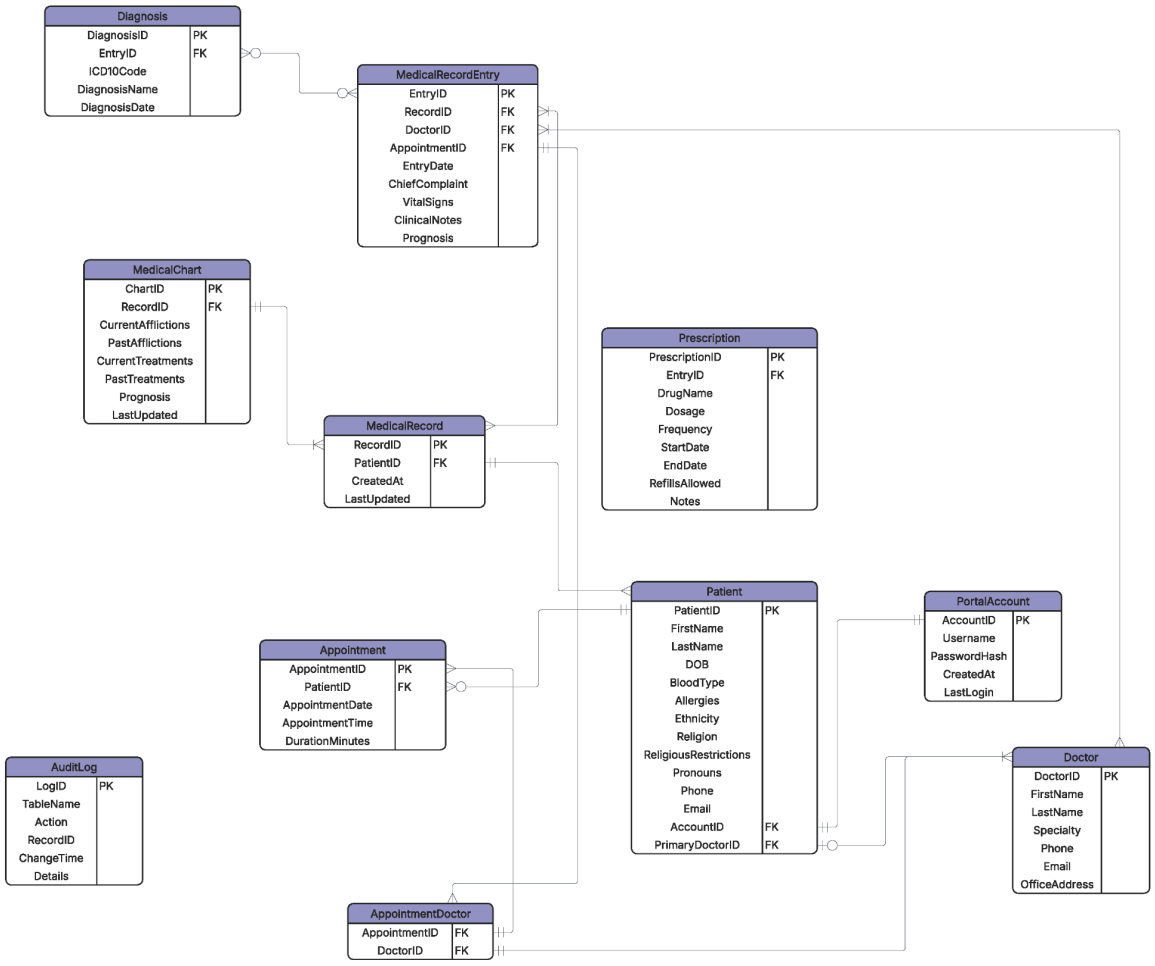
The data was designed to simulate realistic healthcare workflows and support all required queries and automation features. Several doctor and patient names were inspired by fictional media characters for demonstration purposes.

Schema Overview (Next Page)

ERD Diagram


Prescription ID
Entry ID
Drug name
start date
dosage
End date
Prescription
Refills allowed
frequency
notes
Contains
notes
vitals
Entry ID
Entry date
Diagnosis ID
Record ID
prognosis
Diagnosis
Written to
Medical Record Entry
Entry ID
Appt ID
Doctor ID
ICD10 code
complaint
Diagnosis date
Diagnosis name
Contains
Creates
Chart ID
Office Address
Email
Record ID
RecordID
Phone
MedicalChart
Contains
MedicalRecord
patient ID
Specialty
Current Afflictions
created at
Doctor
Last name
Current treatments
Last Updated
Prognosis
last updated
First name
past treatments
Doctor ID
has
has
attends
last login
Appt ID
created at
Portal Account
has
Patient
makes
appointment
Password Hash
Patient ID
Patient ID
Last Name
Religion
Username
First Name
Allergies
Ethnicity
DOB
Restrictions
duration
appt date
Blood Type
Pronouns
Phone
appt time
Email
Account ID
Doctor ID
Account ID

Relational Schema



Normalization (3NF)

- The database follows Third Normal Form (3NF).

First Normal Form (1NF)

- Each table contains atomic values only.
- No repeating groups are present.

Second Normal Form (2NF)

- All non-key attributes depend fully on the entire primary key.
- Partial dependencies were eliminated.

Third Normal Form (3NF)

- Non-key attributes depend only on the primary key.
- Transitive dependencies were removed.
- Separate tables were created for appointments, diagnoses, prescriptions, and medical records to reduce redundancy and improve maintainability.
- This normalization improves consistency, scalability, and query efficiency.

Queries & Automation

- In this section, we demonstrated ten SQL queries and three triggers that show the main functionality of our Patient Chart Portal database. These queries prove that our database can retrieve, organize, summarize, and connect patient, doctor, appointment, diagnosis, and prescription information.
- Query 1 counts the number of appointments by status using COUNT, GROUP BY, and ORDER BY. This helps show how many appointments are completed, scheduled, or cancelled.
- Query 2 shows each doctor and the number of patients assigned to them as their primary doctor. This query uses a LEFT JOIN, which is useful because it still shows doctors even if they have no assigned patients.
- Query 3 lists all appointments with the patient name, doctor name, appointment date, time, reason, and status. This query uses multiple joins and CONCAT to combine first and last names into readable full names.
- Query 4 shows a distinct list of patients who have at least one prescription. The DISTINCT keyword prevents duplicate patients from appearing if they have multiple prescriptions.
- Query 5 finds patients who have more than one diagnosis. This query uses a subquery and COUNT(DISTINCT) to identify patients with multiple medical conditions.
- Query 6 shows the top prescribed medications and how many times each medication was prescribed. This helps identify the most common medications in the system.
- Query 7 lists patients with active or chronic diagnoses along with the doctor connected to their medical record entry. This query is useful because it connects patient conditions to the doctor responsible for care.
- Query 8 calculates the minimum, maximum, and average number of prescriptions per patient. This demonstrates summary statistics using MIN, MAX, AVG, ROUND, and a subquery.
- Query 9 displays full doctor contact and office information. This query is useful for viewing each doctor's specialty, phone number, email, and office address.
- Query 10 displays a full patient demographic and medical background profile. It includes information such as date of birth, blood type, allergies, ethnicity, religion, pronouns, phone number, and primary doctor.
- For automation, our database includes three triggers.
- The first trigger, after_appointment_insert, runs after a new appointment is inserted. It automatically adds a record into the AuditLog table, which helps track new appointment activity.
- The second trigger, after_entry_update, runs after a medical record entry is updated. It automatically updates the LastUpdated field in the related medical record so the system always reflects the most recent change.
- The third trigger, before_patient_delete, runs before a patient is deleted. It prevents a patient from being deleted if they still have scheduled appointments, which protects data integrity and enforces a realistic business rule.

Stored Procedure

The stored procedure, generate_patient_summary, generates a summarized report for a selected patient. It displays active or chronic diagnoses along with current prescriptions that do not have an end date. This procedure simplifies patient reporting and improves efficiency when reviewing important medical information for ongoing treatment.

Function

The user-defined function in the database provides reusable logic that can be called within SQL queries to simplify calculations and improve readability. Functions help reduce repeated code and allow commonly used operations to be performed consistently throughout the database system.